

Final Project Report

Damaged Photo Restoration with a U-Net Autoencoder

Instructor: Dr. Seifipour | Semester 2, 1404–1405

Group member 1	«Abbas Khoshdooni Farahani» — Student ID «810101415»
Group member 2	«Deniz Mostafazadeghan» — Student ID « 810102603»
Group member 3	«Sajjad Taghizade» — Student ID «810102425»
Dataset	joshuachin/openphoto-restore-dataset (Hugging Face Hub)
Task	Supervised image restoration (damaged → clean image pairs)

1. Introduction

Old and damaged photographs commonly suffer from noise, scratches, stains, blur, faded or shifted color, and loss of fine detail. This project addresses the task of restoring a damaged photograph to a version that is closer to its original, undamaged appearance. We treat this as a supervised image-to-image translation problem: a model receives a damaged image and is trained to output a clean image that matches a paired ground-truth image, using the [joshuachin/openphoto-restore-dataset](#) from the Hugging Face Hub.

We selected a U-Net architecture for this task. The report below documents the preprocessing pipeline, the model architecture and training configuration, the quantitative and qualitative results obtained on the held-out test set, and a discussion of the model's limitations and directions for improvement.

2. Preprocessing

Each sample in the dataset consists of a damaged image and its corresponding clean ground-truth image. The following preprocessing pipeline was applied identically to both images in a pair, so that they remain pixel-aligned:

- Resize both the damaged and clean images to a fixed resolution of 256×256 pixels, so that all samples in a batch have matching tensor shapes and training time is kept manageable.
- Convert each image to a PyTorch tensor and scale pixel values from the [0, 255] integer range to a normalized [0, 1] floating-point range.
- Split the data into training, validation, and test sets. The dataset's own train/test split was used as the base; 10% of the training split was further held out as a validation set (4,050 train / 450 validation / 500 test images).
- Wrap the resulting tensors in a custom PyTorch Dataset and load them through a DataLoader for shuffled, batched training (batch size 16).

3. Model Architecture

We implemented a U-Net, a convolutional encoder–decoder architecture originally developed for biomedical image segmentation and widely adopted for image restoration and translation tasks. It was chosen over a plain Autoencoder because of its skip connections, and over a VAE or GAN for the reasons summarized in the comparison below.

3.1 Why U-Net

- Skip connections directly link each encoder stage to its corresponding decoder stage, letting fine spatial detail (edges, texture) bypass the bottleneck instead of being forced through a single compressed representation — this is essential for restoration, where the output must stay pixel-aligned with the input.
- It is fully convolutional and deterministic, so it trains stably with a simple pixel-wise loss and does not require the adversarial training loop of a GAN or the probabilistic latent sampling of a VAE.
- It performs well even with a moderately sized dataset (a few thousand paired images), which matches the scale of data available here.

3.2 Structure

The implemented network follows the standard U-Net design:

- Encoder: 4 downsampling stages, each a pair of 3×3 convolutions (with batch normalization and ReLU) followed by 2×2 max-pooling; the channel width doubles at each stage (64 → 128 → 256 → 512).
- Bottleneck: a further double-convolution block at the deepest, most compressed resolution (1024 channels).
- Decoder: 4 upsampling stages using transposed convolutions; at each stage the upsampled feature map is concatenated with the matching encoder feature map (the skip connection) before another double-convolution block.
- Output head: a 1×1 convolution back to 3 channels, followed by a sigmoid activation so the output is directly comparable to the [0, 1]-normalized ground-truth image.

In total the network has approximately 31.0 million trainable parameters.

3.3 Advantages and limitations for this task

- Advantage: skip connections preserve structural detail, which is reflected in the strong SSIM score obtained (Section 5).
- Advantage: simple, stable training — no adversarial or variational training tricks required.
- Limitation: with a purely pixel-wise loss, the mapping from a damaged (often desaturated) input to a fully colored ground truth is ill-posed — many plausible colorizations exist for the same input — which biases the model toward safe, averaged colors rather than vivid, committed ones (discussed further in Section 7).

4. Loss Function and Training Configuration

The network was trained end-to-end to directly minimize the pixel-wise difference between its output and the ground-truth clean image. We used the L1 (mean absolute error) loss rather than MSE, since L1 tends to produce sharper restorations with less over-smoothing.

Loss function	L1 loss (mean absolute error)
Optimizer	Adam
Learning rate	1e-4
Batch size	16
Number of epochs	30
Input image size	256 × 256
Hardware used	GPU (CUDA), Google Colab
Train / Val / Test split	4,050 / 450 / 500 images

5. Training and Validation Loss

Training and validation loss were logged after every epoch. Both curves decrease steadily and track each other closely throughout training, indicating that the model is learning a generalizable mapping rather than overfitting to the training set; the single spike in validation loss around epoch 18 corresponds to a noisy batch rather than a systematic issue.

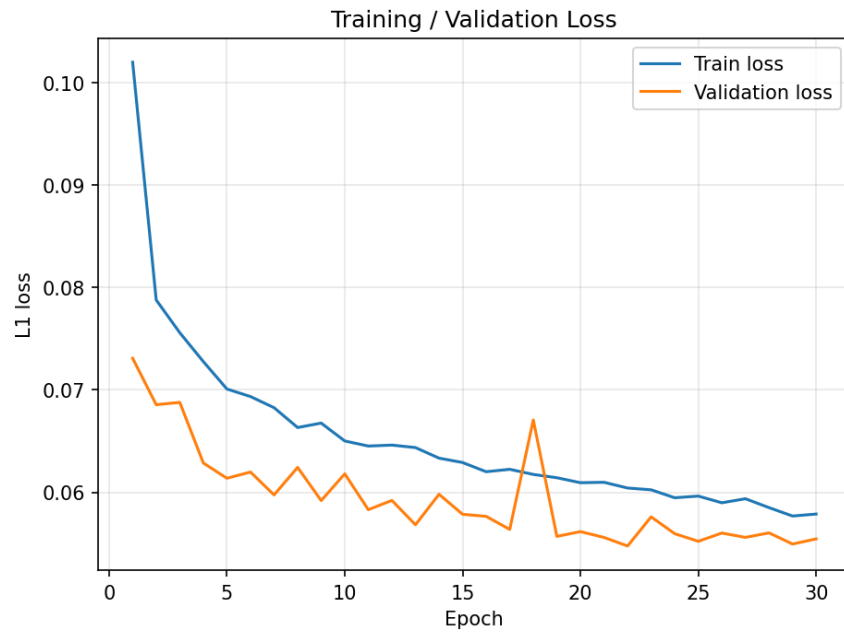


Figure 1. Training and validation L1 loss over 30 epochs.

6. Quantitative Results

The trained model (best checkpoint by validation loss) was evaluated on the 500-image held-out test set using four metrics: mean squared error (MSE), mean absolute error (MAE), peak signal-to-noise ratio (PSNR), and structural similarity (SSIM).

Model	MSE ↓	MAE ↓	PSNR (dB) ↑	SSIM ↑
U-Net (test set)	0.0080	0.0579	22.99	0.863
Minimum required	—	—	≥ 20.00	≥ 0.650

The model exceeds both minimum performance requirements set for the project: PSNR of 22.99 dB against a required 20 dB, and SSIM of 0.863 against a required 0.65.

7. Sample Outputs

Five test examples are shown below, each with the damaged input, the ground-truth clean image, and the model's restored output, side by side.

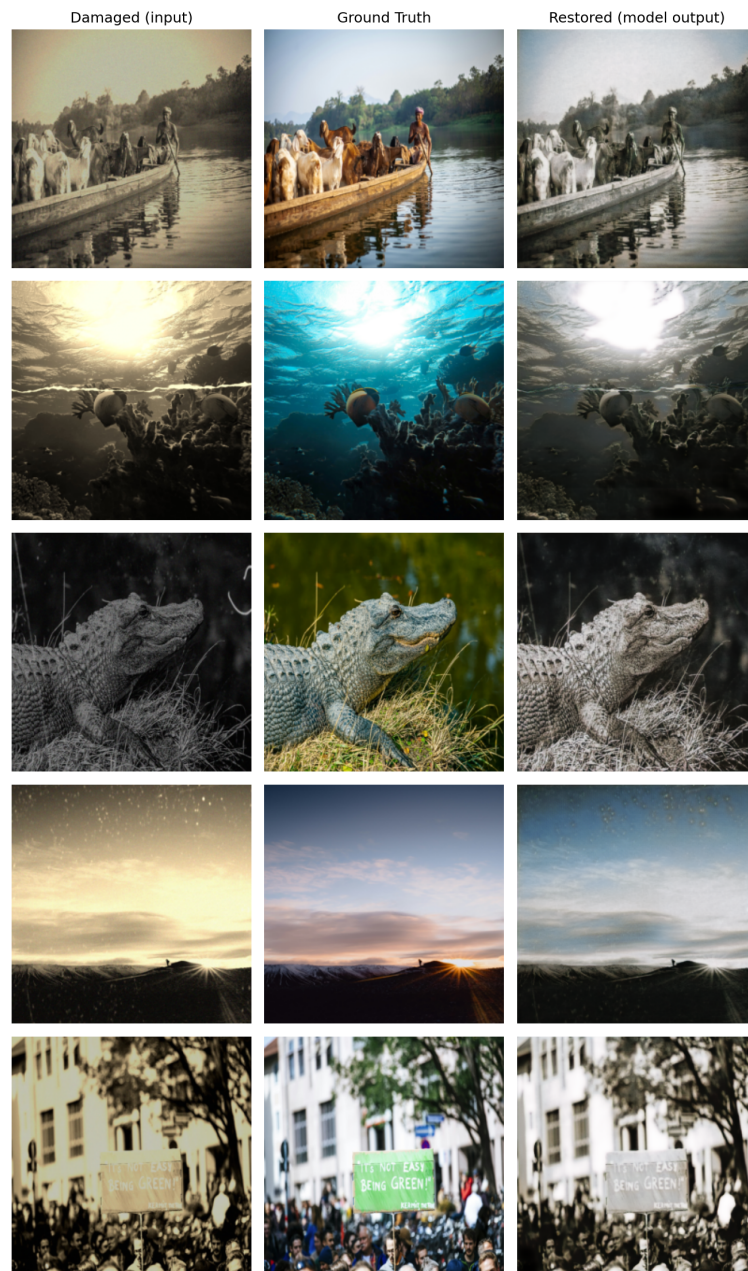


Figure 2. Damaged input / ground truth / model output for five test images.

8. Analysis of Results

Quantitatively, the model performs well above the required thresholds, and the loss curves show stable, well-behaved convergence with no evidence of overfitting. Qualitatively, the model is clearly effective at the tasks most directly rewarded by a pixel-wise loss: it removes speckle noise, softens scratches and stains, and reconstructs plausible fine structure (fur, scales, foliage, architectural edges) that is barely visible in the damaged input — visible for example in the crocodile and boat/goat examples in Figure 2.

The clearest weakness is color restoration. In most examples the model's output stays closer to the muted, desaturated tone of the damaged input than to the vivid color of the ground truth (e.g. the underwater and sunset examples). This is consistent with the known behavior of pixel-wise losses (L1/MSE) on ill-posed reconstruction problems: when many different colorizations are equally plausible given a desaturated input, minimizing average pixel error rewards the model for predicting a safe, averaged color rather than committing to a vivid but potentially 'wrong' one. Structural restoration (shapes, edges, textures) does not have this ambiguity to the same degree, which is why SSIM — which is more sensitive to structure than to exact color — comes out much stronger relative to its threshold than color reproduction would suggest.

9. Limitations and Suggestions for Improvement

9.1 Limitations

- Color restoration is weak / desaturated relative to the ground truth, for the reasons discussed in Section 8.
- A single, deterministic L1 loss cannot express the many plausible outputs that could correspond to one damaged input, so the model regresses toward an average rather than a specific plausible reconstruction.
- The model was evaluated only with full-reference metrics (PSNR, SSIM, MSE, MAE); no qualitative/expert evaluation was performed, which the project brief also allows for as a complementary evaluation method.
- Training and evaluation were done at a fixed 256×256 resolution; performance on higher-resolution originals is untested.

9.2 Suggestions for improvement

- Add an adversarial loss term (a lightweight GAN discriminator on top of the existing U-Net generator) or a perceptual loss (e.g. VGG feature loss) alongside the L1 loss, to push the model toward sharper, more saturated, more 'committed' outputs instead of averaged ones.
- Apply data augmentation (random crops, flips, color jitter) to increase effective dataset diversity and improve robustness.
- Train for more epochs with a learning-rate schedule (e.g. cosine decay or reduce-on-plateau) now that the current run shows the loss was still slowly decreasing at epoch 30.
- Experiment with a deeper or wider U-Net, or a pretrained encoder backbone, to improve capacity, and validate at higher input resolutions.
- Collect qualitative feedback from human raters, as suggested in the project brief, to complement the pixel-based metrics with a perceptual judgment of restoration quality.